DAY 2 · MASTER PRESENTATION

Functions, College OOP Architecture, Error Resilience & Git

Building Reusable Functions, Simulating KIT College with Advanced OOP (USN, MRO, Polymorphism, Abstraction) & Version Control

Welcome to Day 2! Here is What You and I Will Master Today:

- Section 3: Functions as reusable machines, *args/**kwargs, execution time decorators, and lambdas.
- Section 4: Advanced OOP simulated around KIT College: USN standard, Abstraction (ABC), Encapsulation (@property), and Name Mangling.
- Section 4: 5 Types of Inheritance, MRO (C3 Linearization), Campus Hierarchy (Management, Support, Faculty), Polymorphism & Operator Overloading.
- Section 4 & 6: Defensive Error Handling (try/except/else/finally), leak-proof file streams, and Git/GitHub collaboration.
- Interactive Lab: 4 comprehensive Day-2 Jupyter Notebooks (.ipynb) with live runnable code!

DAY 2 FLIGHT PLAN

The 4 Power Pillars of Today's Journey

From writing modular functions to simulating our college campus and sharing code on GitHub

PILLAR 1

Modular Functions

- Anatomy: Inputs, body, return
- Dynamic *args & **kwargs
- Stopwatch @decorators
- Fast anonymous lambdas
- Built-in Python modules

PILLAR 2

KIT College OOP

- Class (blueprint) vs Object (heap)
- Abstraction with abc.ABC
- Encapsulation & private __cgpa
- @property getter & setter
- USN collegiate standard

PILLAR 3

Advanced OOP & MRO

- 5 Types of Inheritance
- MRO & C3 Linearization
- Management & Support staff
- Dynamic Polymorphism
- Operator Overloading (+, >, ==)

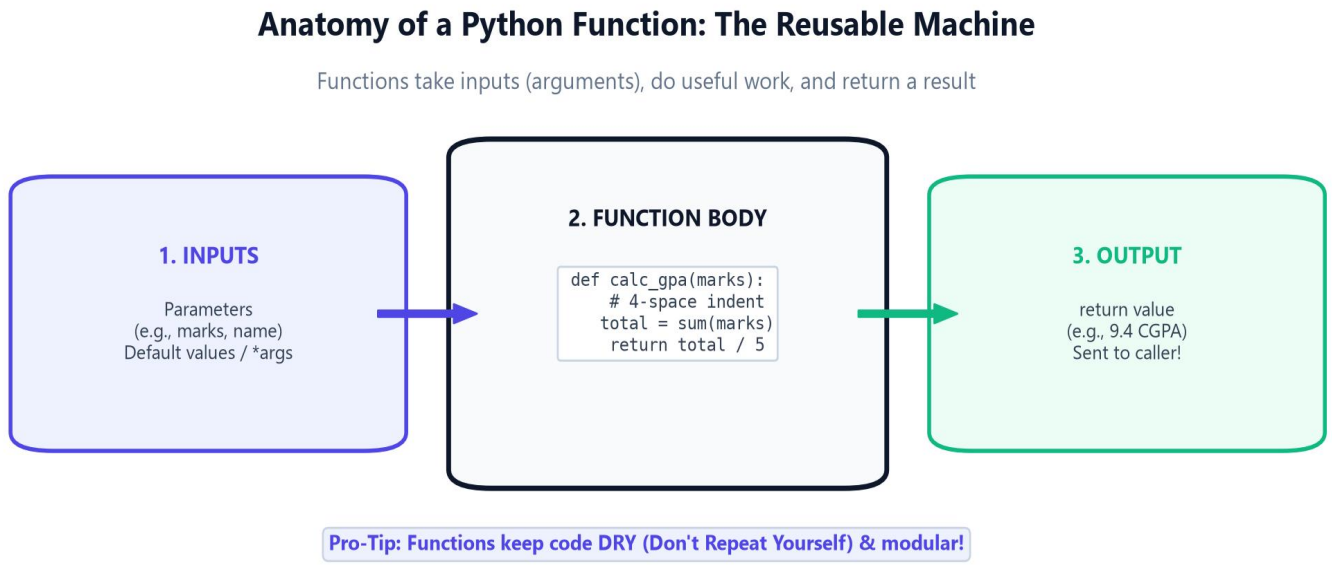
PILLAR 4

Errors & Git Highway

- 4-stage try/except/else/finally
- Safe with open() file streams
- Reading & writing CSV rosters
- Git 4 zones: Add, Commit, Push
- AI .gitignore & Pull Requests

Anatomy of a Function: The Reusable Machine

Functions take inputs, perform operations, and return outputs — keeping your code DRY



Default Arguments Rule:

You can provide sensible default values (e.g., def calc_fee(amount, college='KIT'):). If callers don't pass the college, it automatically uses 'KIT'!

Key Function Concepts:

1. def keyword: Signals to Python that you are defining a reusable function.
2. Parameters: Variables listed inside parentheses that receive values from caller.
3. Indented Body: 4 spaces define the scope of what the function executes.
4. return statement: Sends computed results back; without return, Python gives back None!
5. Docstrings: Triple quotes """ explain what your function does for team documentation.

Dynamic Arguments (*args, **kwargs) & Decorators

Handling arbitrary electives and wrapping functions with timing stopwatches

1. Positional *args

Packs into a Tuple: *args gathers extra unnamed parameters.

Variable Electives: enroll_student('Sundeep', 'AI', 'NLP', 'Robotics')

Unpacking: *my_list expands items into function arguments effortlessly.

2. Keyword **kwargs

Packs into a Dict: **kwargs gathers named keyword parameters.

Flexible Profiles: register(hostel='Block A', scholarship=50)

Forwarding: Passes unknown hyperparameters directly to underlying AI models.

3. The @ Decorator

The Campus Guard: Wraps around your function to add logging or timing.

@track_time: Starts a stopwatch before the function runs and reports milliseconds after it finishes!

@wraps: Preserves original function name and docstrings.

HANDS-ON PRACTICE

Let's Open Notebook 01: Functions & Modules

Time to run campus GPA calculations, decorators, and leaderboard sorting live!

Open: notebooks/day_02/01_day2_functions_and_modules.ipynb

1. Open Notebook 01 and run the calculate_gpa() semester grade function.
2. Test register_kit_student() using *electives and **extra_details with USN 1KT23AI042.
3. Run the @track_execution_time stopwatch decorator on an AI training simulation.
4. Use a 1-line lambda function to rank KIT student exam scores descending.
5. Complete the attendance eligibility challenge and verify your result!

What is a Class & Object? The Memory Model

Understanding the blueprint vs the living instance allocated in heap memory

1. Class vs Object

The Blueprint (Class): Code defining attributes and methods. Cost = 0 bytes in heap until instantiated.

The Instance (Object): Real entity loaded in RAM: s1 = Student('Sundeep', '1KT23AI001').

Analogy: Class is the blank KIT admission form. Object is YOU enrolled!

2. Memory Inspection

id(obj): Unique RAM address in heap (e.g. 0x0000021A5).

type(obj): Points to the blueprint class definition.

obj.__dict__: Python stores every object's instance state in a native hashmap dictionary!

3. Class vs Instance Scope

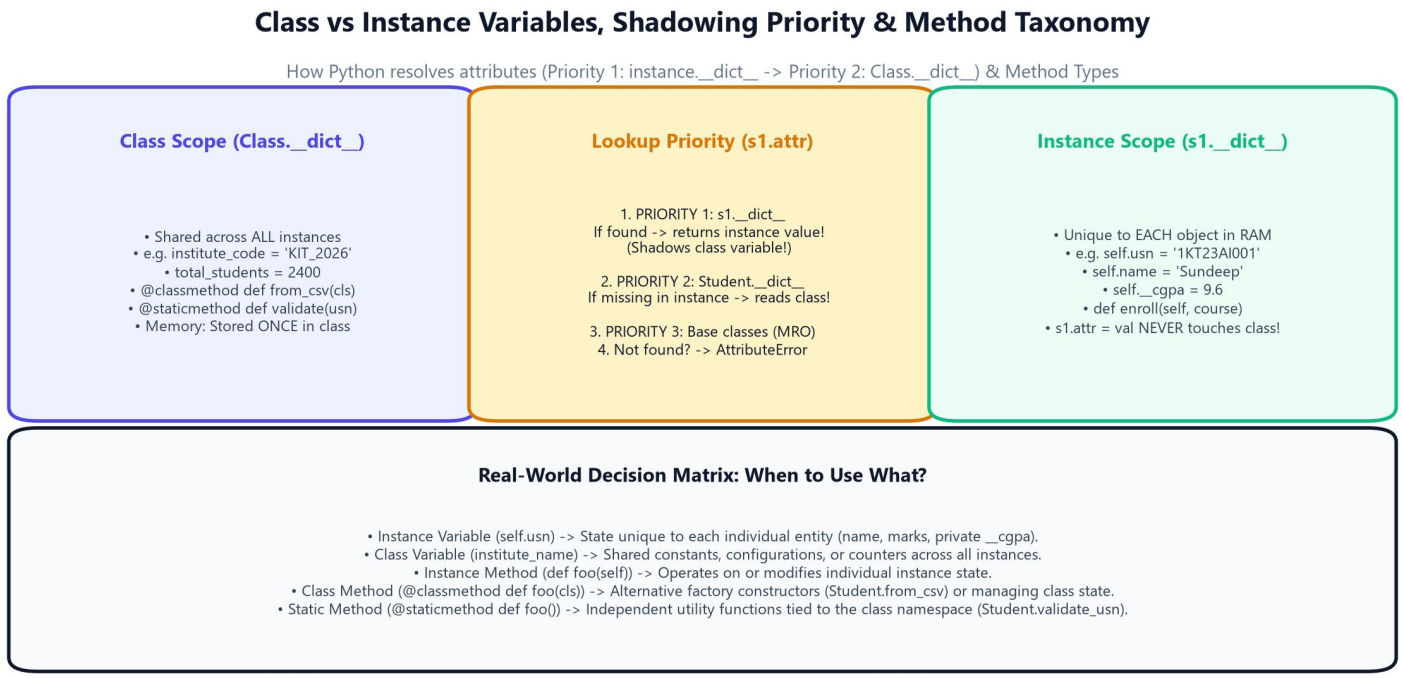
Class Attributes: institute_name = 'KIT College Institute'. Shared across all 2,000 students!

Instance Attributes: self.name and self.usn are unique to each student.

Memory Efficiency: Shared methods and class variables save megabytes of RAM.

Class vs Instance Scope: Lookup Priority & Methods

Understanding attribute shadowing in __dict__, the mutable trap, and self vs cls vs @staticmethod



IITian Rule of Thumb — Where to Use What:

If state belongs to one entity -> Instance. If state belongs to the blueprint -> Class. If logic needs no state -> Static utility.

Attribute Priority & Taxonomy:

- Priority 1 (Instance): Python checks instance.__dict__ first. If present, it returns immediately!
- Priority 2 (Class): If not in instance, Python checks Class.__dict__, then MRO parent classes.
- Shadowing Trap: s1.campus = 'North' writes to s1.__dict__. It NEVER modifies KITStudent.campus!
- Mutable Class Trap: clubs = [] at class level is shared by ALL students! Always initialize inside __init__.
- Instance Method: Takes self. Accesses unique instance state (e.g. s1.record_exam_score()).
- Class Method: Takes cls. Modifies class state or acts as factory constructor (from_csv).
- Static Method: Takes no self/cls. Pure isolated utility function (validate_usn).

Abstraction: Enforcing Mandatory Campus Interfaces

Using abc.ABC and @abstractmethod to guarantee that incomplete entities cannot exist

1. What is Abstraction?

Hiding Complexity: Users only see WHAT an object does, not HOW it works internally.

Campus Contract: The governing board declares: every campus member MUST have a role summary and payout formula!

Design First: Architects define blueprints before engineers build concrete subclasses.

2. abc.ABC & @abstractmethod

from abc import ABC: Inherit from ABC to mark class as an abstract base.

@abstractmethod: Decorate methods that subclasses are legally REQUIRED to implement.

Contract Enforcement: If a subclass forgets to implement an abstract method, Python blocks it!

3. Direct Instantiation Shield

Can't Instantiate Ghost: CollegeMember('Ghost') raises TypeError immediately.

Error Message: "Can't instantiate abstract class with abstract methods get_role_summary"

Bug Elimination: Eliminates half-baked objects before your app ever reaches production.

Encapsulation & Access Modifiers: Public, Protected & Private

How Python protects sensitive grades and why 'We are all consenting adults'

1. Public & Protected (_)

Public (self.usn): Freely accessible anywhere across campus programs.

Protected (self._conduct): Single underscore prefix. Developer convention: 'Internal use only; do not tamper!'

Subclass Friendly: Protected variables are easily inherited by child classes.

2. Private (__) & Name Mangling

Double Underscore: __cgpa and __salary trigger Name Mangling.

Accidental Block: student.__cgpa from outside raises AttributeError.

Behind the Scenes: Python renames it to _Student__cgpa in RAM to prevent accidental overrides!

3. 'Consenting Adults' Rule

Guido's Philosophy: Python does not build impenetrable C++ walls.

Trust the Developer: You CAN access student._Student__cgpa, but it violates the API contract!

Standard Practice: Use @property getters and setters for all controlled data.

The Gatekeeper: @property Getter, Setter & Deleter

Writing clean attribute access while strictly validating grades and sensitive records

1. @property Getter

Clean Syntax: Write print(student.cgpa) without ugly parentheses .get_cgpa()!

Read-Only Shield: Omit the setter to make any attribute permanently immutable.

Computed Fields: Can compute student standing or honors status dynamically on read.

2. @cgpa.setter Guardrail

Strict Validation: if 0.0 <= new_grade <= 10.0: save it!

Crash Prevention: Raises ValueError if someone inputs negative or impossible marks (e.g. 15.0).

Enterprise Standard: Identical to how Pydantic and PyTorch validate inputs.

3. @cgpa.deleter Action

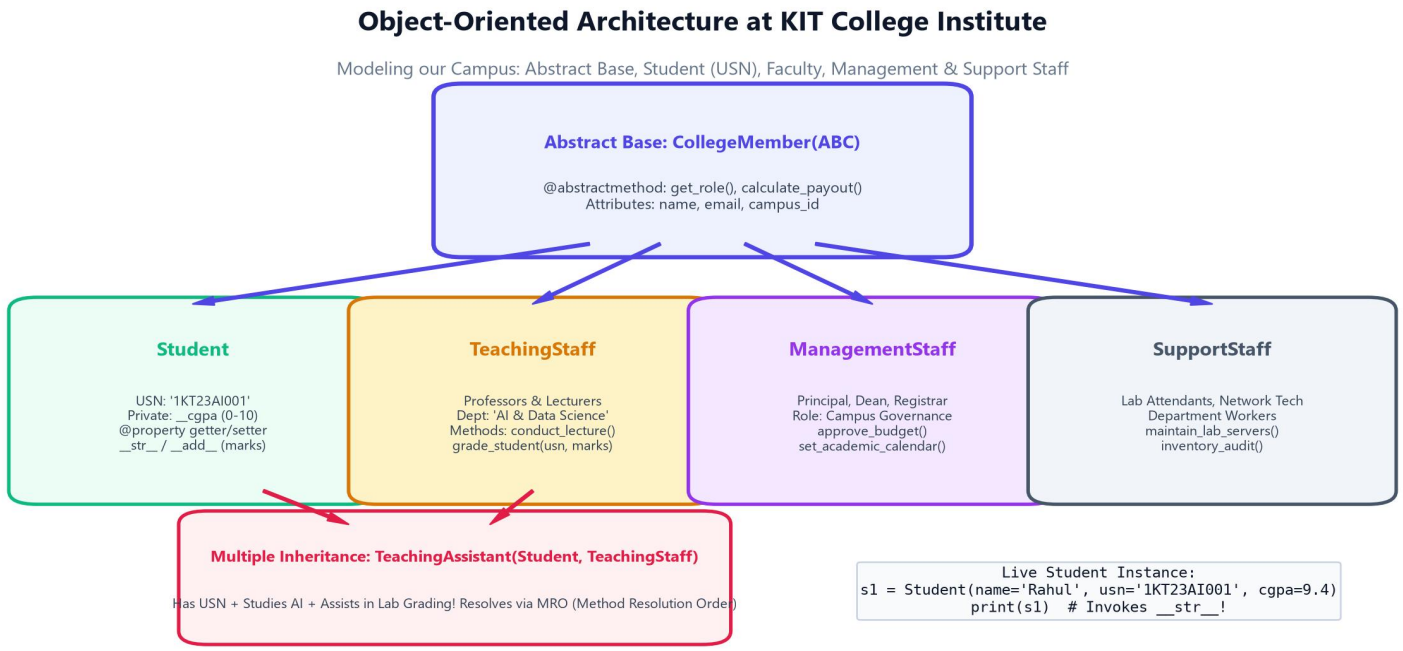
del student.cgpa: Controls what happens when an attribute deletion is attempted.

Safe Reset: Instead of deleting records, resets grade to 0.0 with a security log!

Audit Trail: Logs who triggered the deletion and when.

The KIT College Model: Complete Entity Hierarchy

Modeling our campus: Abstract Base, Student (USN), Faculty, Management & Support Staff



The USN Standard at KIT:

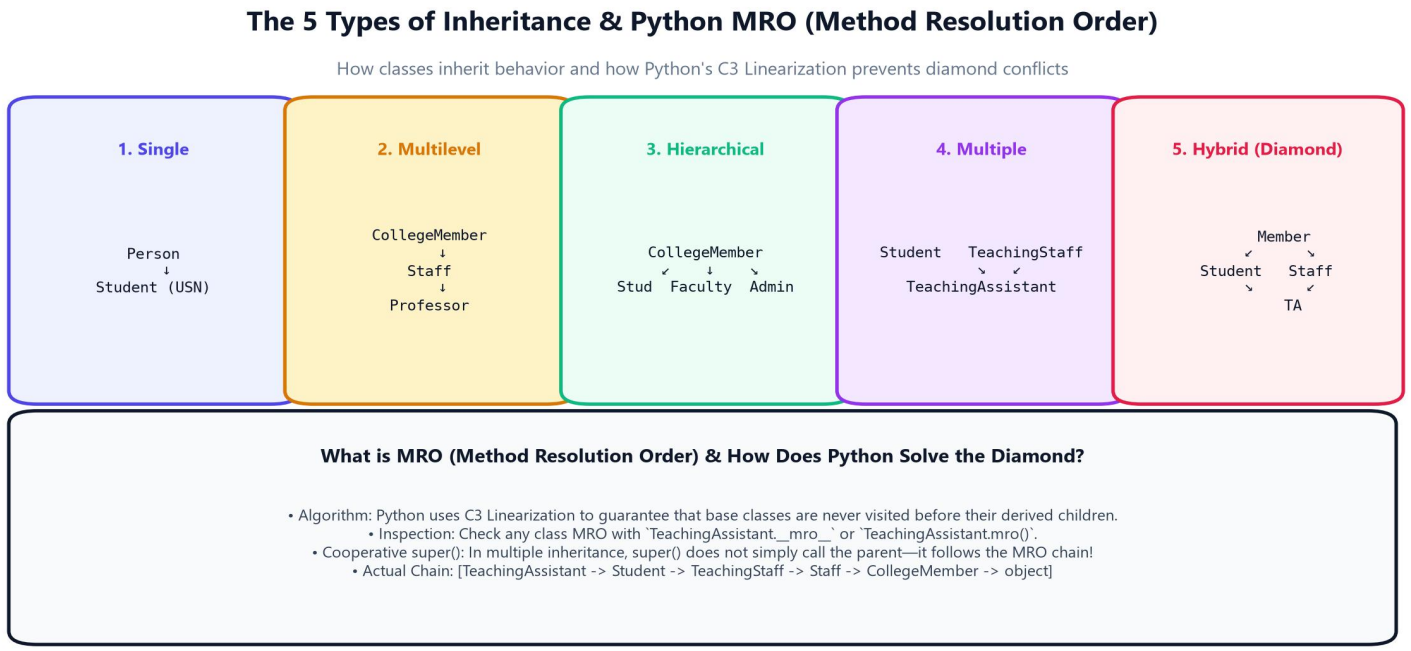
In college, roll numbers are replaced by official University Seat Numbers (USN: 1KT23AI001). Every student record is uniquely mapped to their USN!

Campus Hierarchy Overview:

- CollegeMember(ABC): Abstract base class requiring get_role_summary() & calculate_payout().
- Student: Has USN (e.g. '1KT23AI001'), branch, private __cgpa, and merit payouts.
- TeachingStaff: Professors conducting lectures and research.
- ManagementStaff: Principal & Deans approving departmental budgets.
- SupportStaff: AI Lab Attendants servicing GPU racks.
- TeachingAssistant: Multiple inheritance: Student + TeachingStaff!

The 5 Types of Inheritance in Python

Single, Multilevel, Hierarchical, Multiple, and Hybrid (Diamond) Architectures



Why Multiple Inheritance Matters:

In Python, a Teaching Assistant is genuinely both a Student (takes exams) and an Employee (grades lab sessions). Multiple inheritance captures this duality!

Real Campus Examples:

1. Single: Person -> Student (One parent to one child).
2. Multilevel: CollegeMember -> Staff -> TeachingStaff (A chain of inheritance).
3. Hierarchical: CollegeMember -> Student, Staff, Management (One parent, many siblings).
4. Multiple: TeachingAssistant(Student, TeachingStaff) (Two parents!).
5. Hybrid / Diamond: TA connects back to CollegeMember via two separate branches!

MRO (Method Resolution Order) & C3 Linearization

How Python resolves diamond inheritance without duplicate method calls or ambiguities

1. The Diamond Problem

Two Parent Paths: Both Student and Staff inherit from CollegeMember.

Ambiguity Hazard: If both override get_role(), which one should TA execute?

C++ Flaw: In naive languages, the base constructor runs twice! Python solves this.

2. C3 Linearization Algorithm

Strict Ordering: Children appear before parents; parent order in class def is preserved.

No Duplicates: Guarantees every base class in the tree is visited exactly once!

Inspection: Check any class with TeachingAssistant.__mro__ or .mro().

3. Cooperative super()

Not Just Parent: super() doesn't call the immediate parent—it follows the MRO chain!

Chain Order: [TA -> Student -> TeachingStaff -> Staff -> CollegeMember -> object]

Seamless Coordination: Ensures all initializers and methods coordinate harmoniously.

4.3A · CAMPUS OPERATIONS

Campus Roles: Management Staff & Support Staff

Empowering the entire institution: Leadership, Professors, Lab Technicians & Attendants

1. Management Staff

Campus Leadership: Principal, Deans, Registrar running administration.

Specific Methods: approve_budget('AI Lab', 850000.0) and set_academic_calendar().

Governance: Ensures college accreditation and institutional operations run smoothly.

2. Support Staff & Workers

Department Backbone: AI Lab Attendants, GPU Engineers, Network Techs.

Specific Methods: perform_maintenance('NVIDIA RTX 4090 Rack') and lab_inventory_audit().

Operational Health: Keeps high-performance AI compute clusters running 24/7.

3. Teaching Assistant (TA)

Hybrid Scholar: Senior student who studies and instructs lab sections.

Dual Role: Enjoys student merit scholarships + employee monthly stipend.

Lab Leadership: ta_meera.conduct_lecture('YOLOv8 Computer Vision Lab')!

Polymorphism: Dynamic Duck Typing vs Static Overloading

Why Python has no method signature overloading and how we solve it with modern patterns

1. Dynamic Polymorphism

Duck Typing: 'If it walks like a duck and quacks like a duck, it's a duck!'

Unified Loop: for m in campus: print(m.get_role_summary())

No Type Checks: Zero need for ugly if isinstance(...) checks; Python dispatches at runtime automatically!

2. Static Overloading Truth

No Signature Overloading: In Java, you can have 2 methods with different parameter types.

Python Overwrite: In Python, defining def foo() a second time completely REPLACES the first!

Why?: Python is dynamically typed; signatures are checked at call time, not compile time.

3. Python's Overloading Solutions

Default Arguments: def register(usn, hostel=None, scholarship=0)

Arbitrary Arguments: Use *args and **kwargs to handle variable parameters.

@singledispatch: Built-in functools module dispatches based on the type of the first argument!

4.3A · OPERATOR MAGIC

Operator Overloading: Custom Math & Logic Operators

Teaching +, >, <, and == to interact seamlessly with our custom campus objects

1. Arithmetic: __add__ (+)

Math on Objects: Enables writing student1 + student2!

Summing Credits: Combines exam marks or team project credit scores.

Fallback: Return NotImplemented if the other operand type is unsupported.

2. Comparisons: __gt__ & __lt__

Direct Ranking: Enables s1 > s2 based on academic CGPA.

sorted() Integration: Implementing __lt__ allows Python's sorted(students) to work automatically!

Clean Analytics: Sort thousands of student records in 1 line of code.

3. Equality: __eq__ (==)

By Value, Not Pointer: Default == checks if memory addresses match. That's usually not what we want!

USN Matching: __eq__ checks if self.usn == other.usn.

Duplicate Detection: Identifies duplicate student entries across campus databases instantly.

Dunder Methods: __str__ vs __repr__ & Container Protocols

Making custom classes native: Human report cards, developer debugging, and department containers

1. __str__ vs __repr__

__str__ (Human): Clean, friendly view for students & parents: '🎓KIT Student: Sundeep | USN: 1KT23AI001'.

__repr__ (Dev Debug): Unambiguous developer view: Student(name='Sundeep', usn='1KT23AI001', cgpa=9.6).

Golden Rule: __repr__ should ideally produce a string that can recreate the object in code!

2. __len__ & __getitem__

len(department): Returns enrolled student count without writing dept.get_count().

department[0]: Accesses first student by integer index.

department['1KT23AI001']: Accesses student directly by USN string through bracket syntax!

3. __contains__ & __iter__

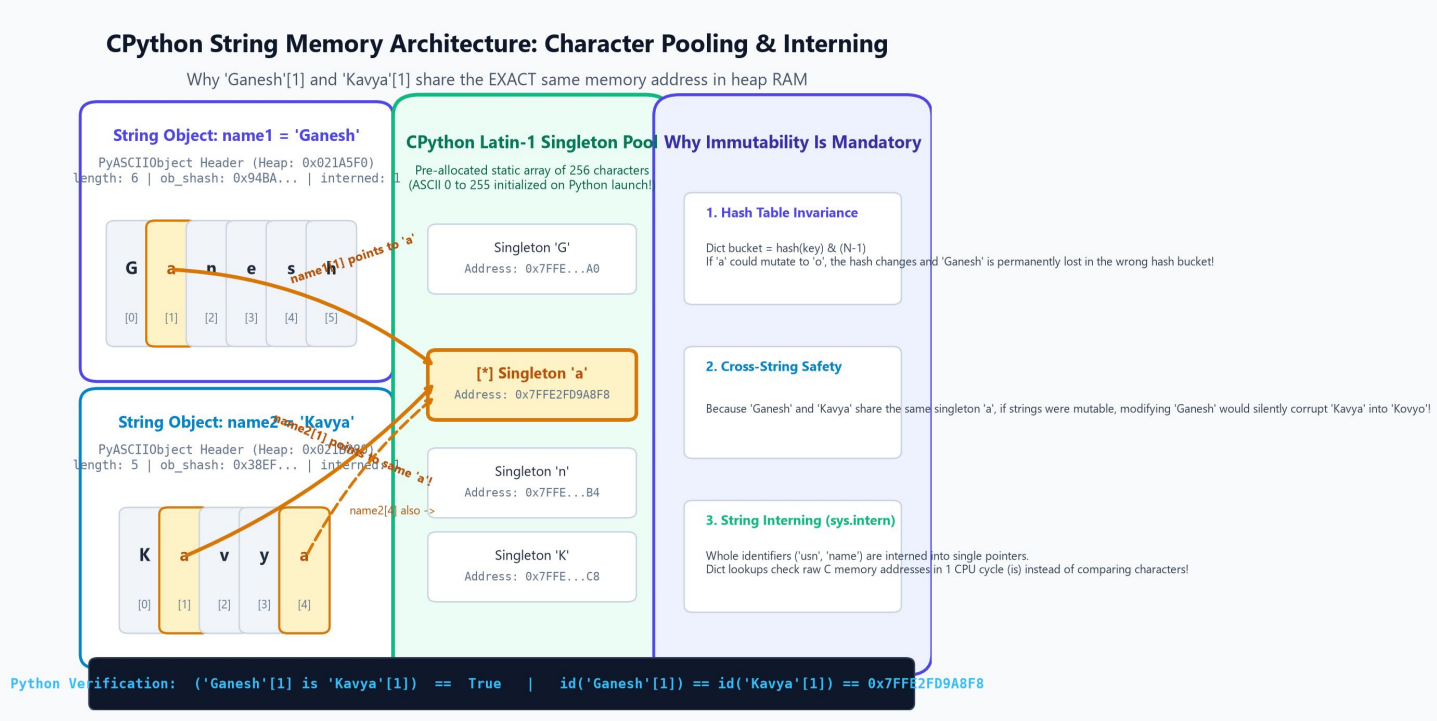
'1KT23AI001' in dept: Enables lightning-fast membership checking using the 'in' keyword!

for student in dept: __iter__ makes the department iterable in standard for loops.

PyTorch Connection: This exact container protocol powers PyTorch Dataset and DataLoader!

String Internals, Immutability & The Hash Contract

Why strings are immutable, how hash tables achieve O(1) speed, and CPython string interning



Python Live Memory Verification:

('Ganesh'[1] is 'Kavya'[1]) == True | id('Ganesh'[1]) == id('Kavya'[1]) == 0x7FFE2FD9A8F8 | Singletons shared across RAM!

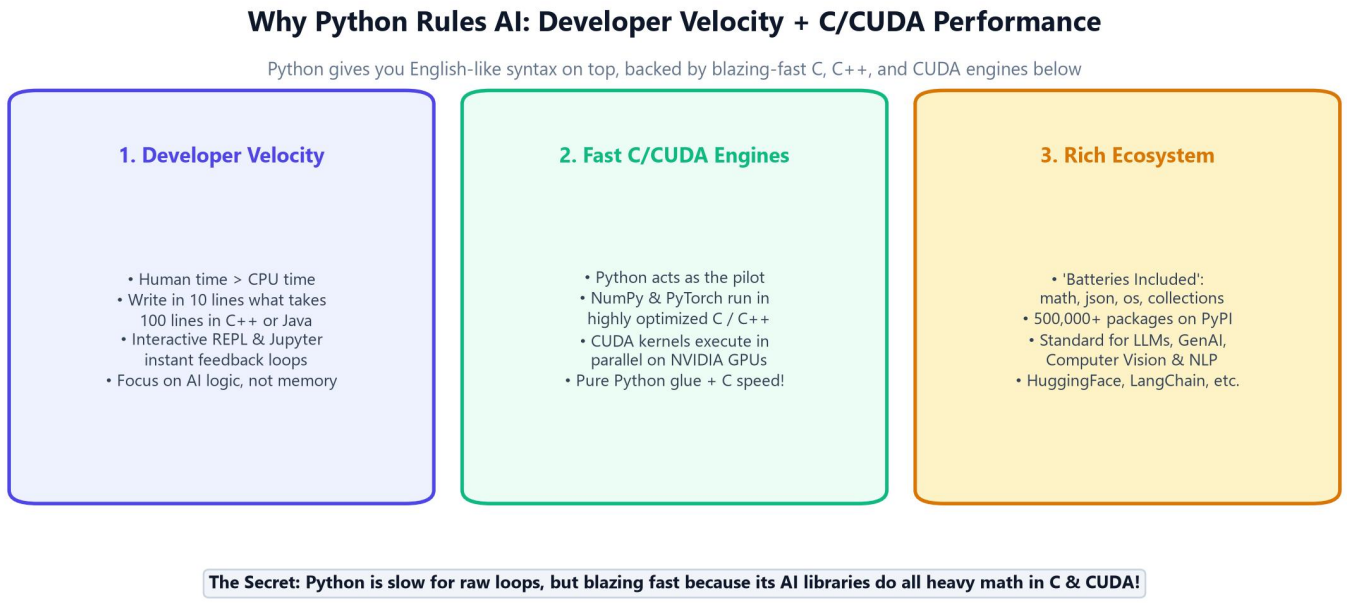
Character Pooling & Hash Invariance:

- Singleton Pooling: 'Ganesh'[1] and 'Kavya'[1] share the EXACT same heap pointer (0x7FFE2FD9A8F8) in the 256 Latin-1 singleton pool!
- Hash Contract: a == b => hash(a) == hash(b). Overriding __eq__ disables __hash__. Explicitly implement __hash__ on classes to enable set() and dict keys.
- String Interning: sys.intern() ensures identical identifiers share 1 RAM address. Dictionary lookups become 1-cycle pointer equality checks!
- Why Immutability?: If 'a' could mutate to 'o', 'Ganesh' would mutate to 'Gonesh' AND 'Kavya' would mutate to 'Kovyo'! Plus, hash changes break dict buckets permanently.
- SipHash-2-4 Defense: Random PYTHONHASHSEED salting eliminates HashDoS algorithmic complexity attacks on web servers.

INDUSTRY INSIGHTS

Why Python Rules AI: Developer Velocity + C/CUDA Engines

High-level English readability on top, powered by blistering C and NVIDIA CUDA kernels below



Vectorization Speedup Benchmark:

In Notebook 02, we benchmark 1,000,000 score calculations: Pure Python takes ~85 ms, while NumPy takes ~0.8 ms (100x speedup!) due to underlying C vectorization.

The AI Developer Advantage:

- Developer Velocity: Human engineering time is expensive; cloud GPU time is cheap. Python writes in 10 lines what takes 100 in Java.
- C Engine Secret: PyTorch, NumPy, and TensorFlow execute in compiled C/C++ and CUDA.
- Best of Both Worlds: Python serves as the steering wheel for high-performance C++ Formula 1 engines!
- Batteries Included: Built-in math, json, os, datetime, and collections.
- PyPI Ecosystem: 500,000+ packages: HuggingFace, LangChain, Transformers, OpenCV.

HANDS-ON PRACTICE

Let's Open Notebook 02: KIT College OOP Master Simulation

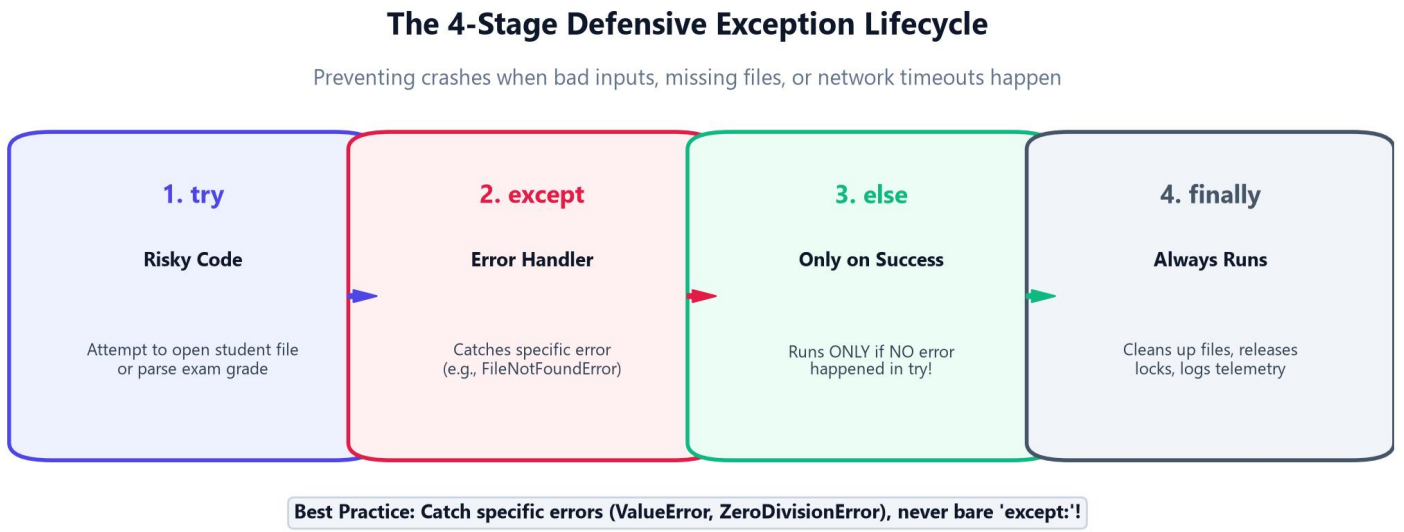
Instantiate yourself with a USN, test MRO, override operators, and manage your department!

Open: notebooks/day_02/02_day2_college_oops_kit_institute.ipynb

1. Open Notebook 02 and verify that instantiating CollegeMember(ABC) directly raises TypeError.
2. Create Student Sundeep with USN '1KT23AI001' and test private __cgpa guardrails.
3. Inspect TeachingAssistant.__mro__ and watch C3 Linearization resolve diamond inheritance.
4. Test operator overloading: Add marks with +, sort students with >, and check USN equality with ==.
5. Populate KITDepartment and access students via index and USN string (ai_dept['1KT23AI001'])!
6. Run the live NumPy vs Pure Python benchmark to witness the 100x C-vectorization speedup!

The 4-Stage Defensive Exception Lifecycle

How we ensure our software never crashes when bad inputs, missing files, or network timeouts happen



Production Exception Rule:

Never silence errors with 'except: pass'. That creates invisible bugs that corrupt databases and AI pipelines. Always log the error or re-raise with context!

The 4 Golden Stages:

1. try: Place only the risky code here (opening files, network requests, math division).

2. except ErrorType: Catches specific problems gracefully. Never use bare except:!

3. else: Runs ONLY if the try block completed with zero errors. Keeps success logic clean.

4. finally: Runs ALWAYS, whether an error occurred or not! Perfect for closing database connections.

Built-In vs Custom Domain Exceptions

Catching specific errors and defining college business rule exceptions like AttendanceShortageError

1. Common Built-In Errors

ValueError: Right data type, but illegal value (e.g., CGPA = 15.0).

FileNotFoundError: Trying to read a dataset that doesn't exist on disk.

ZeroDivisionError: Dividing by zero in average calculation.

KeyError: Searching for a student USN missing from dictionary.

2. Custom Domain Exceptions

class AttendanceShortageError(Exception): Subclass from Python's base Exception.

Domain Clarity: Signals domain-specific business rules to callers.

raise keyword: raise AttendanceShortageError(f'{usn} has only {pct}% attendance!')

3. Defensive AI Engineering

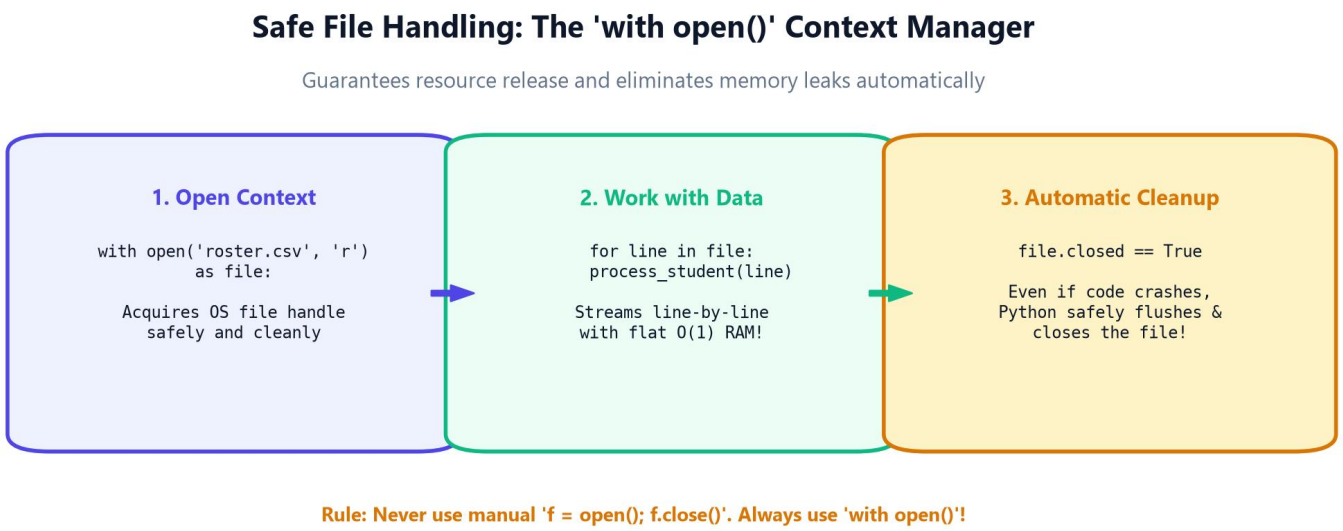
API Rate Limits: Catch 429 TooManyRequests and retry with exponential backoff.

Context Length: Catch TokenLimitExceeded and automatically truncate prompt history.

Model Failovers: Fall back to a local model if the cloud API times out.

Safe File Handling: The 'with open()' Context Manager

Automatic resource acquisition and cleanup that eliminates file handle leaks permanently



The Golden Rule of File I/O:

Never write manual 'f = open(...)' and 'f.close()'. If an error happens before close(), the OS file lock leaks. 'with open()' guarantees cleanup 100% of the time!

File Modes Guide:

'r' (Read): Default mode. Raises FileNotFoundError if file is missing.

'w' (Write): Overwrites existing file completely or creates a new one.

'a' (Append): Adds new lines at the end without erasing previous data.

encoding='utf-8': Always specify utf-8 to avoid international encoding bugs!

line streaming: for line in file: streams massive gigabyte files with O(1) RAM!

Processing Tabular Data: CSV & JSON Streams

Reading student records, serializing model configurations, and managing AI datasets

1. Writing CSVs with csv.writer

import csv: Python's standard library module for tabular spreadsheets.

writer.writerow([headers]): Writes USN, Name, Branch, CGPA.

writer.writerows(rows): Dumps complete student batches in one shot with automatic escaping.

2. Reading with csv.DictReader

Dict Mapping: Turns every CSV row into a clean dictionary {'USN': '1KT23AI001'}!

Header Resilient: Code doesn't break if column order in the CSV changes.

Flat Memory Usage: Streams row-by-row without loading the whole file into RAM.

3. JSON Configurations

json.dump(data, file): Saves Python dicts directly as structured JSON files.

json.load(file): Loads hyperparameters and student profiles back into Python.

Universal Standard: The universal format for REST APIs, LLM prompts, and tool calling.

HANDS-ON PRACTICE

Let's Open Notebook 03: Errors & Safe File Handling

Test defensive error recovery and generate student CSV rosters on disk!

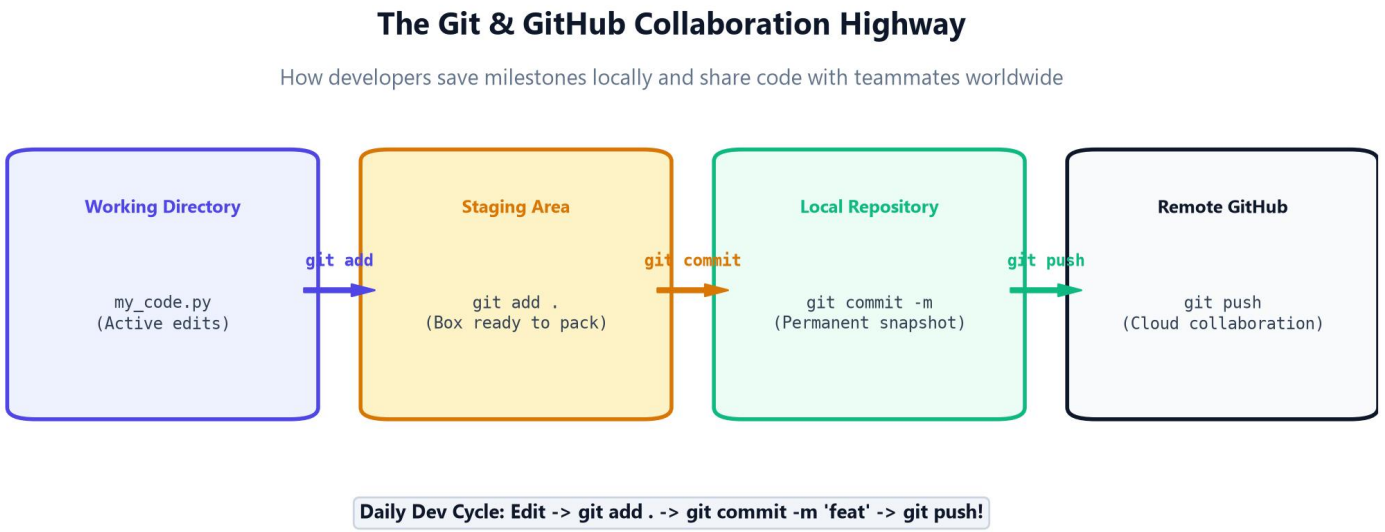
Open: notebooks/day_02/03_day2_errors_and_file_handling.ipynb

1. Open Notebook 03 and run the exam score parser with valid, invalid, and string inputs.
2. Observe how the finally block executes every single time for audit telemetry.
3. Write a batch of KIT student records to 'kit_students_roster.csv' using USN.
4. Stream the CSV records back using csv.DictReader and filter for honors students (CGPA >= 9.0).
5. Complete the missing file error recovery challenge and verify that your script doesn't crash!

6.1A · COLLABORATION HIGHWAY

The Git & GitHub Workflow Highway

How modern developers track milestones locally and collaborate across teams globally



Git Mental Model:

Git is not just backup; it is a time machine! Every commit is a checkpoint you can return to if a new experiment breaks your model.

The 4 Git Zones:

1. Working Directory: Your active folder where you edit Python code.

2. Staging Area: The packaging zone (git add .). Choose exactly what to include.

3. Local Repository: Permanent snapshot with author & message (git commit -m).

4. Remote GitHub: Cloud repository for team collaboration and backup (git push).

6.1A · PROFESSIONAL WORKFLOW

The Daily Git Cycle & Feature Branching

Writing clean commit messages, inspecting changes, and creating safe development branches

1. Daily Command Cycle

git status: Checks modified files in your working directory.

git add .: Stages all modified files for the next commit.

git commit -m 'feat': Creates permanent snapshot with a descriptive message.

git push origin main: Uploads local commits to GitHub cloud.

2. Safe Feature Branching

Never code on main: Production code lives on main; new features get their own branch.

git checkout -b feat/ai: Creates and switches to a new feature branch.

git branch -a: Lists all local and remote branches.

git merge feat/ai: Merges tested feature into main.

3. Inspection & History

git log --oneline: Compact visual view of commit history.

git diff: Shows exact line-by-line additions and deletions.

git restore file.py: Discards unwanted uncommitted local changes instantly.

6.1B · OPEN SOURCE STANDARDS

Production .gitignore & The Pull Request (PR) Highway

Protecting API secrets, preventing multi-gigabyte model commits, and collaborating on GitHub

1. Essential AI .gitignore

.env (CRITICAL): Never commit API keys or database passwords!

*.pt / *.onnx / *.safetensors: Large model weights belong on HuggingFace or S3, not in Git!

__pycache__/ & *.pyc: Compiled Python bytecodes.

.venv/ & env/: Virtual environments.

2. The Pull Request (PR)

Propose Changes: Push feature branch to GitHub and open a Pull Request.

Peer Review: Teammates review code diffs, comment on lines, and request tweaks.

CI/CD Automation: Automated GitHub Actions run pytest test suites on every PR.

3. Open Source Collaboration

Forking: Create your personal copy of an open source repo on GitHub.

Upstream Sync: Keep your fork updated with the main project.

Merge & Deploy: Once approved, PR merges into main and triggers automatic deployment!

HANDS-ON PRACTICE

Let's Open Notebook 04: Git & GitHub Live Repo

Initialize a Git repository, write an AI .gitignore, make commits, and simulate branches!

Open: notebooks/day_02/04_day2_git_and_github_essentials.ipynb

1. Open Notebook 04 and initialize a test Git repository using Python's subprocess.
2. Create a production .gitignore file blocking .env, large model weights, and __pycache__.
3. Stage student scripts and make a clean semantic commit: 'feat: add kit college student model'.
4. Create a feature branch 'feat/rag-pipeline' and verify its isolation from main.
5. Inspect git log --oneline to view your permanent repository timeline!

DAY 2 GRADUATION

Master Summary: You are Now an Object-Oriented AI Developer!

Congratulations on mastering functions, campus architecture, MRO, errors, and Git collaboration

Functions & Modules

- Modular reusable logic
- *args & **kwargs flexibility
- Stopwatch @decorators
- 1-line lambda functions

KIT College OOP

- Classes vs Objects in RAM
- Abstraction (abc.ABC)
- Encapsulation & private __cgpa
- USN: 1KT23AI001 standard

Advanced OOP & MRO

- 5 Types of Inheritance
- MRO C3 Linearization
- Campus Hierarchy (Management)
- Dynamic Polymorphism & + > ==

Errors & Git Highway

- 4-stage try/except/else/finally
- with open() context manager
- Git add, commit, push
- PRs & AI .gitignore

What Comes Next in Day 3:

Tomorrow we enter Data Analysis & Numerical Computing with NumPy and Pandas! You will leverage your OOP, error handling, and Git skills to build production data pipelines for Generative AI models.